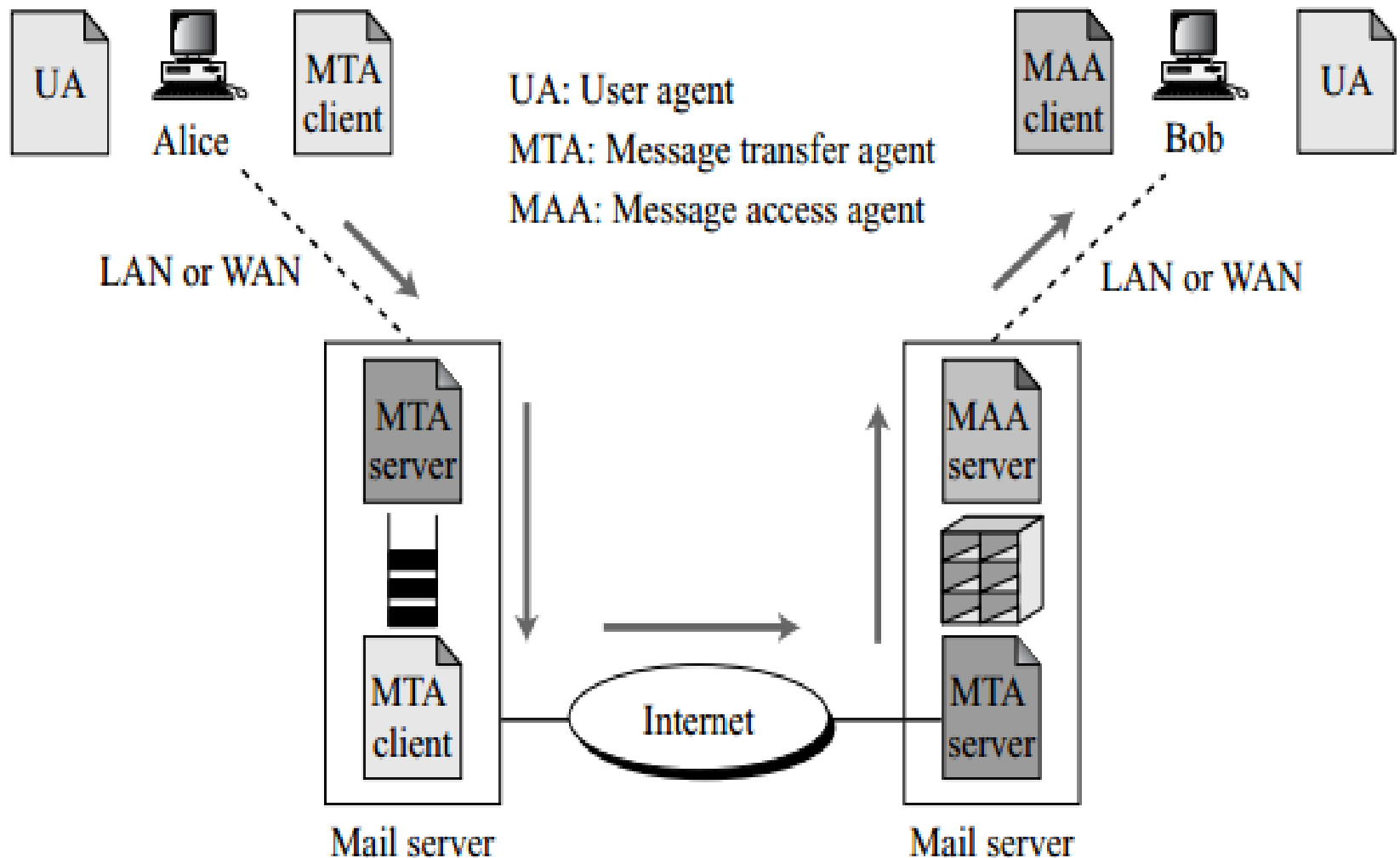


Electronic Mail Security

Electronic Mail Security

- Involves the security at application layer
- Achieved through two protocols:
 - Secure/Multipurpose Internet Mail Extension (S/MIME)
 - Pretty Good Privacy (PGP)

electronic mail (e-mail) Architecture



e-mail Architecture

- The **user agent (UA)** program is used to prepare the email message
- The **message transfer agent (MTA)** is a client/server program with the client installed at Alice's computer and the server installed at the mail server
 - Is used to send the message to the mail server at client site
- The **message access agent (MAA)**
 - designed as a client/server program
 - Is used to retrieve email message

e-mail Architecture

- The sending of an e-mail from Alice to Bob is a store-retrieve activity.
- The main communication between Alice and Bob is through two application programs:
 - the MTA client at Alice's computer
 - the MAA client at Bob's computer

e-mail Architecture

- The MTA client program is a push program
 - the client pushes the message when Alice needs to send it.
- The MAA client program is a pull program; the client pulls the messages when Bob is ready to retrieve his e-mail.
- Alice and Bob cannot directly communicate using an MTA client at the sender site and an MTA server at the receiver site.

e-mail Security challenges

- Sending an e-mail is a one-time activity
- **Challenge 1:**
 - If there is no session and no handshaking to negotiate the algorithms for encryption/decryption and hashing, how can the receiver know which algorithm the sender has chosen for each purpose?

Solution:

In e-mail security, the sender of the message needs to include the name or identifiers of the algorithms used in the message.

e-mail Security challenges

- **Challenge 2:**
- If there is no negotiation, how can the two parties establish secrets between themselves?

Solution:

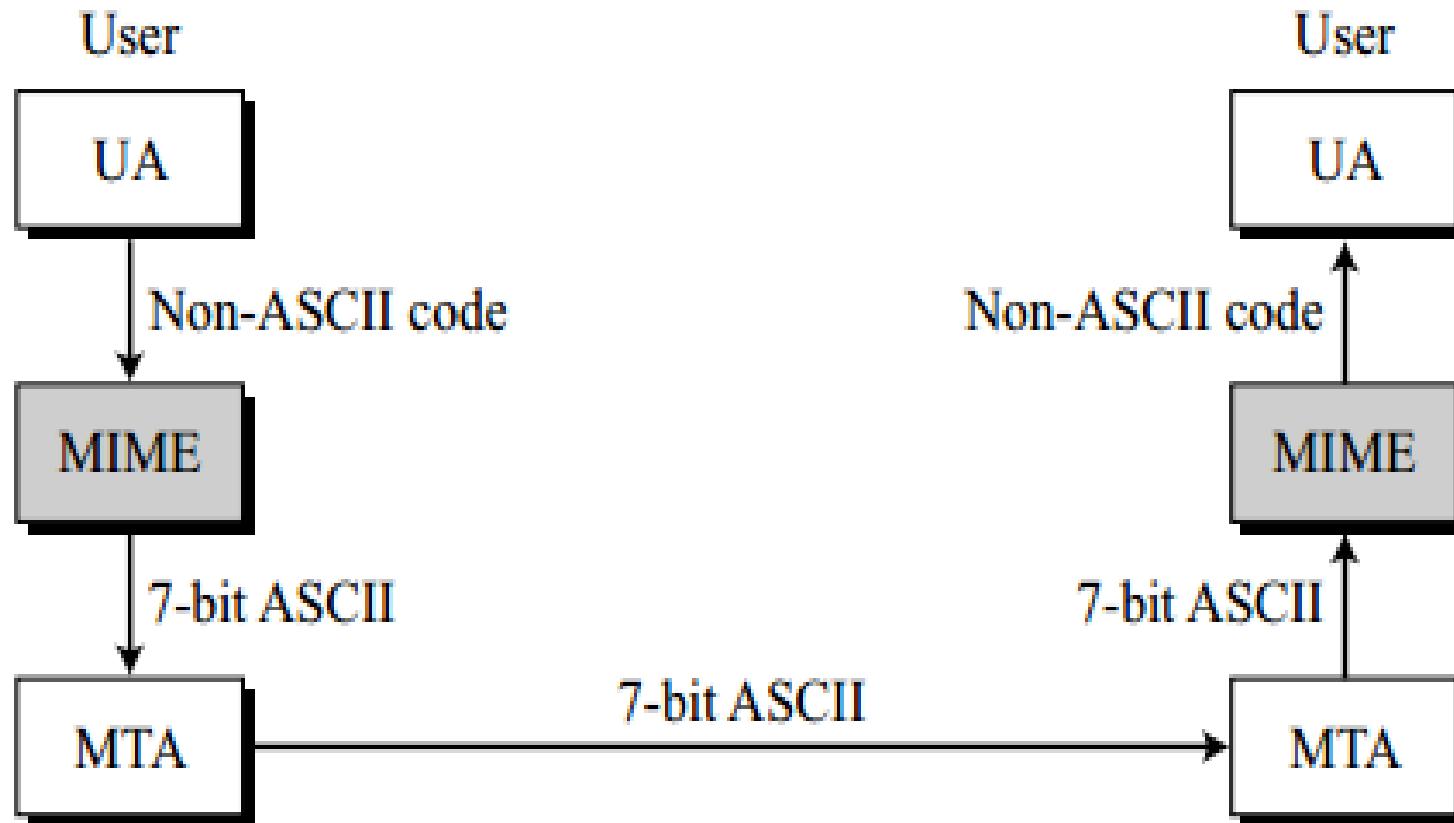
- In e-mail security, the encryption/decryption is done using a symmetric-key algorithm, but the secret key to decrypt the message is encrypted with the public key of the receiver and is sent with the message.
- public-key algorithms can be used for e-mail security, with appropriate method of certifying keys

Multipurpose Internet Mail Extension (MIME) Protocol

Multipurpose Internet Mail Extension (MIME)

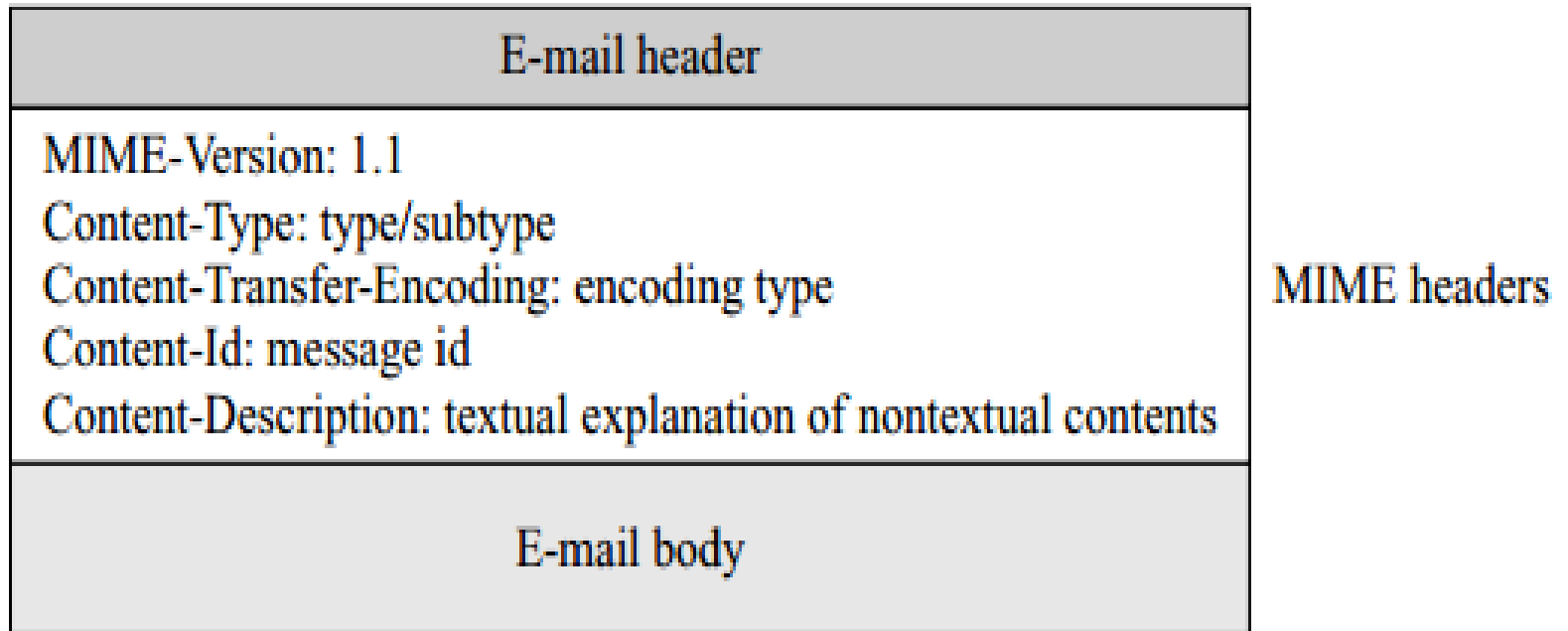
- Electronic mail
 - cannot be used for languages that are not supported by 7-bit ASCII characters
 - cannot be used to send binary files or video or audio data
- MIME is a supplementary protocol that allows non-ASCII data to be sent through e-mail
 - transform non-ASCII data to ASCII data, and vice versa

Multipurpose Internet Mail Extension (MIME)



MIME header

- MIME defines five headers that can be added to the original e-mail header section



Content-Type

- This header defines the type of data used in the body of the message.
- The content type and the content subtype are separated by a slash.
- Depending on the subtype, the header may contain other parameters.

Content-Type: <type / subtype; parameters>

Data types and subtypes in MIME

- MIME allows seven different types of data
 - Text
 - Multipart
 - Message
 - Image
 - Video
 - Audio
 - Application

Data types and subtypes in MIME

-

<i>Type</i>	<i>Subtype</i>	<i>Description</i>
	Plain	Unformatted.
	HTML	HTML format.
Multipart	Mixed	Body contains ordered parts of different data types.
	Parallel	Same as above, but no order.
	Digest	Similar to Mixed, but the default is message/RFC822.
	Alternative	Parts are different versions of the same message.
Message	RFC822	Body is an encapsulated message.
	Partial	Body is a fragment of a bigger message.
	External-Body	Body is a reference to another message.
Image	JPEG	Image is in JPEG format.
	GIF	Image is in GIF format.
Video	MPEG	Video is in MPEG format.
Audio	Basic	Single channel encoding of voice at 8 KHz.
Application	PostScript	Adobe PostScript.
	Octet-stream	General binary data (eight-bit bytes).

Data types and subtypes in MIME

- Text
 - The original message is in 7-bit ASCII format
 - no transformation by MIME is needed
 - There are two subtypes currently used, plain and HTML

Data types and subtypes in MIME

- Multipart
 - The body contains multiple, independent parts
 - A parameter is used to define the boundary between each part
 - The parameter is a string token that comes before each part
 - Four subtypes are defined for this type: [mixed](#), [parallel](#), [digest](#), and [alternative](#)

- Example of a multipart message using a mixed subtype

Content-Type: multipart/mixed; boundary=xxxx

--xxxx

Content-Type: text/plain;

.....

--xxxx

Content-Type: image/gif;

.....

--xxxx--

- Message

- In the message type, the body is itself an entire mail message, a part of a mail message, or a pointer to a message.
- Three subtypes are currently used:
 - RFC822** (if the body is encapsulating another message (including header and the body))
 - partial** (used if the original message has been fragmented into different mail messages)
- The fragments must be reassembled at the destination by MIME.
- Three parameters must be added: **id**, **number**, and the **total**.

- Example of a **message** with three fragments:

```
Content-Type: message/partial;  
id="forouzan@challenger.atc.fhda.edu";  
number=1;  
total=3;
```

```
.....
```

```
.....
```

- The subtype **external-body** indicates that the body does not contain the actual message but is only a reference (pointer) to the original message.

```
Content-Type: message/external-body;  
name="report.txt";  
site="fhda.edu";  
access-type="ftp";
```

```
.....
```

```
.....
```

Content-Transfer-Encoding

- This header defines the method used to encode the messages into 0s and 1s for transport
- There are five types of encoding methods

Content-Transfer-Encoding: <type>

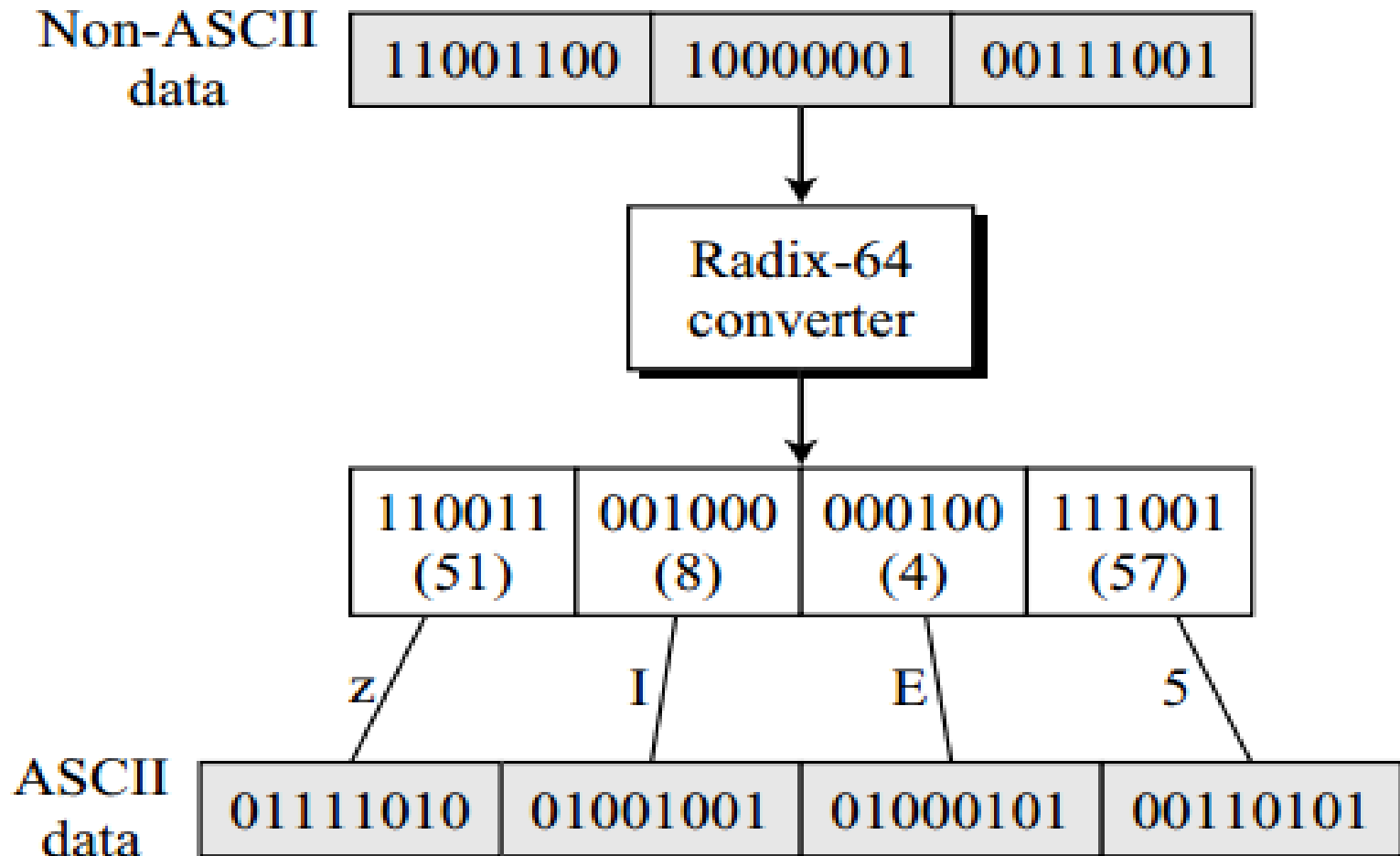
Content-Transfer-Encoding in MIME

<i>Type</i>	<i>Description</i>
7bit	NVT ASCII characters and short lines.
8bit	Non-ASCII characters and short lines.
Binary	Non-ASCII characters with unlimited-length lines.
Radix-64	6-bit blocks of data are encoded into 8-bit ASCII characters using Radix-64 conversion.
Quoted-printable	Non-ASCII characters are encoded as an equal sign followed by an ASCII code.

Radix-64

- Radix-64 divides the binary data (made of streams of bits) into 24-bit blocks.
- Each block is then divided into four sections, each made of 6 bits
- Each 6-bit section is interpreted as one character according to Radix-64 encoding table

Radix-64 conversion



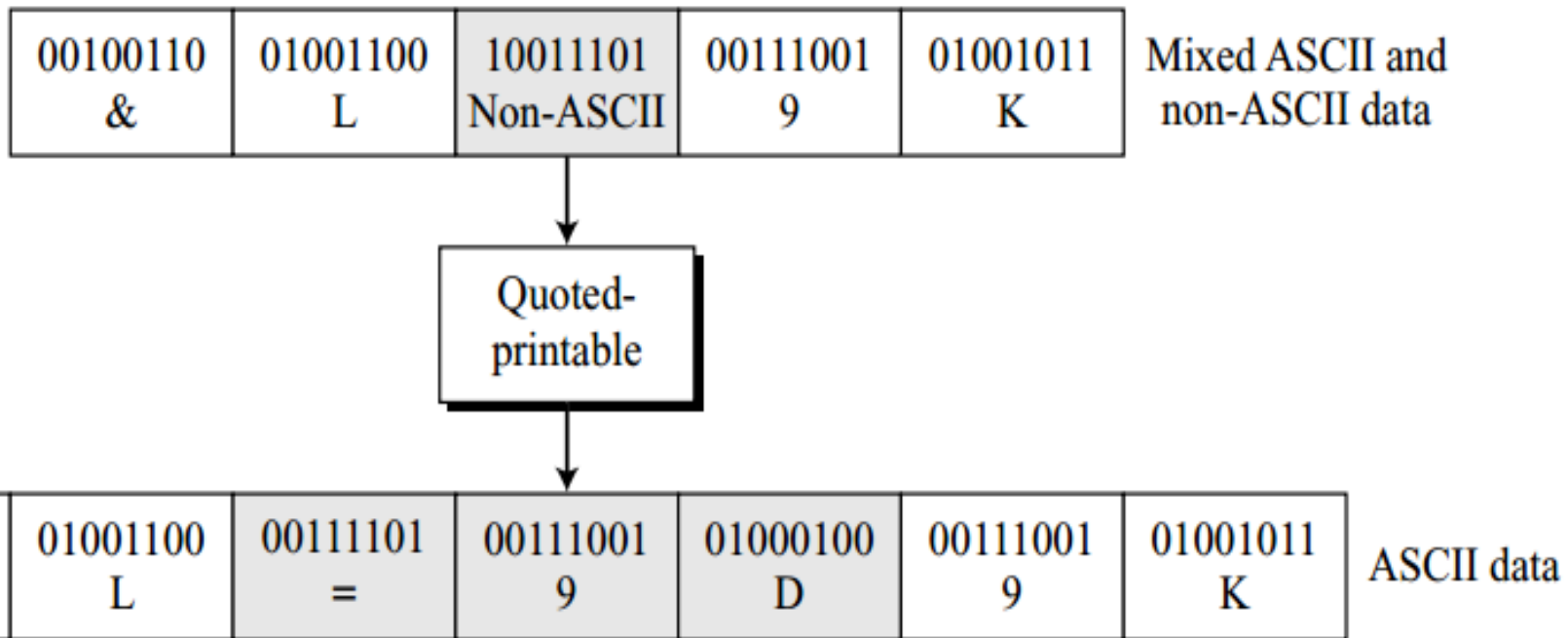
Radix-64 encoding table

<i>Value</i>	<i>Code</i>	<i>Value</i>	<i>Code</i>	<i>Value</i>	<i>Code</i>	<i>Value</i>	<i>Code</i>	<i>Value</i>	<i>Code</i>	<i>Value</i>	<i>Code</i>
0	A	11	L	22	W	33	h	44	s	55	3
1	B	12	M	23	X	34	i	45	t	56	4
2	C	13	N	24	Y	35	j	46	u	57	5
3	D	14	O	25	Z	36	k	47	v	58	6
4	E	15	P	26	a	37	l	48	w	59	7
5	F	16	Q	27	b	38	m	49	x	60	8
6	G	17	R	28	c	39	n	50	y	61	9
7	H	18	S	29	d	40	o	51	z	62	+
8	I	19	T	30	e	41	p	52	0	63	/
9	J	20	U	31	f	42	q	53	1		
10	K	21	V	32	g	43	r	54	2		

Quoted-printable

- **Radix-64** is a redundant encoding scheme
 - 24 bits become four characters and eventually are sent as 32 bits.
- If the data consist mostly of ASCII characters with a small non-ASCII portion, **quoted-printable encoding** can be used.
 - If a character is ASCII, it is sent as is.
 - If a character is not ASCII, it is sent as three characters.
 - The first character is the equal sign (=).
 - The next two characters are the hexadecimal representations of the byte.

Quoted-printable



S/MIME

S/MIME

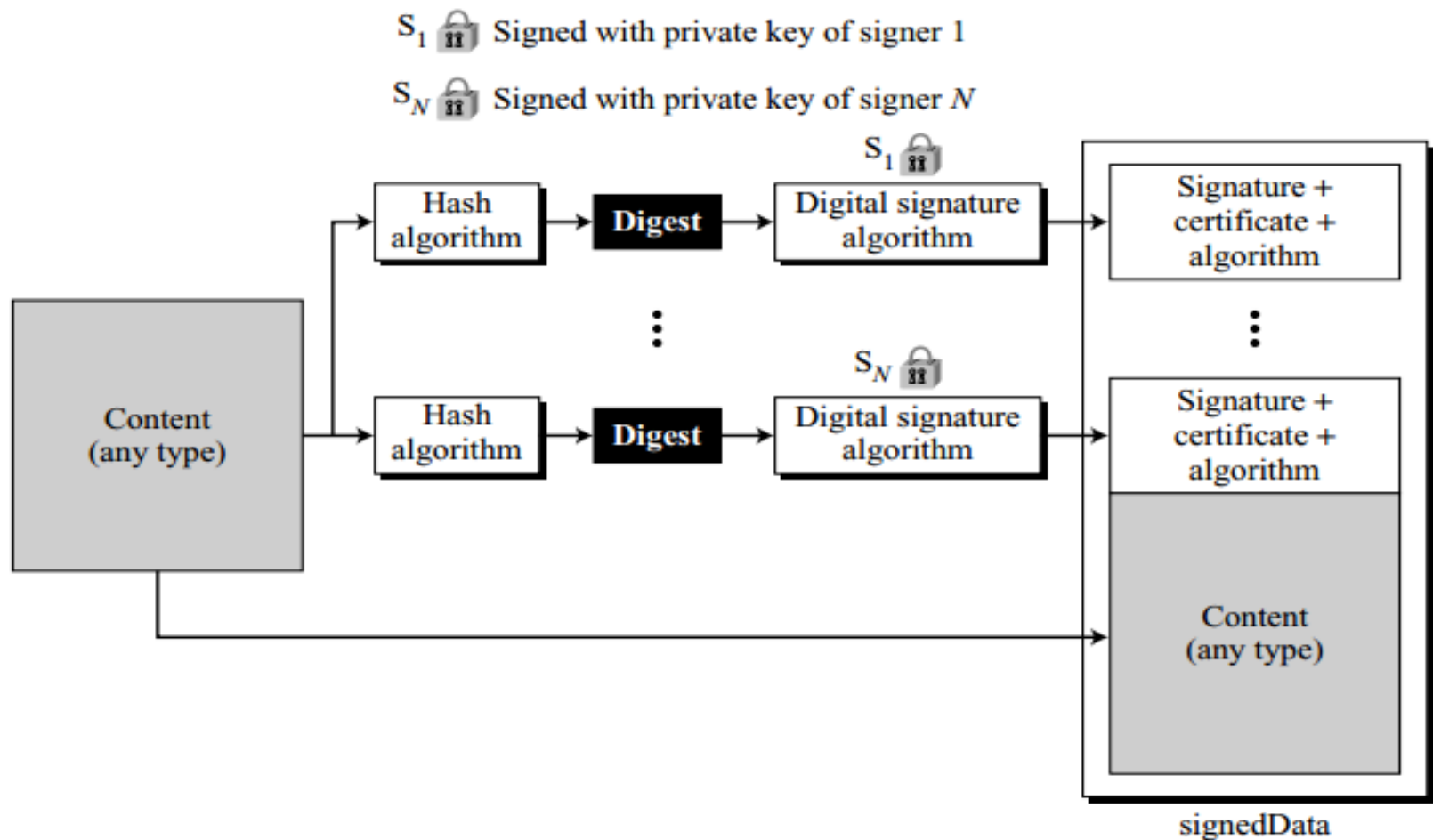
- S/MIME adds some new content types to include **security services** to the MIME
- S/MIME **has defined Cryptographic Message Syntax (CMS)**
 - defines the exact encoding scheme for each content type

Type of message

- **Data Content Type**
 - This is an arbitrary string.
 - The object created is called Data.
- **Signed-Data Content Type**
 - provides only integrity of data
 - It contains any type and zero or more signature values.
 - The encoded result is an object called `signedData`.

Type of message

- Signed-data content type



- **Enveloped-Data Content Type**

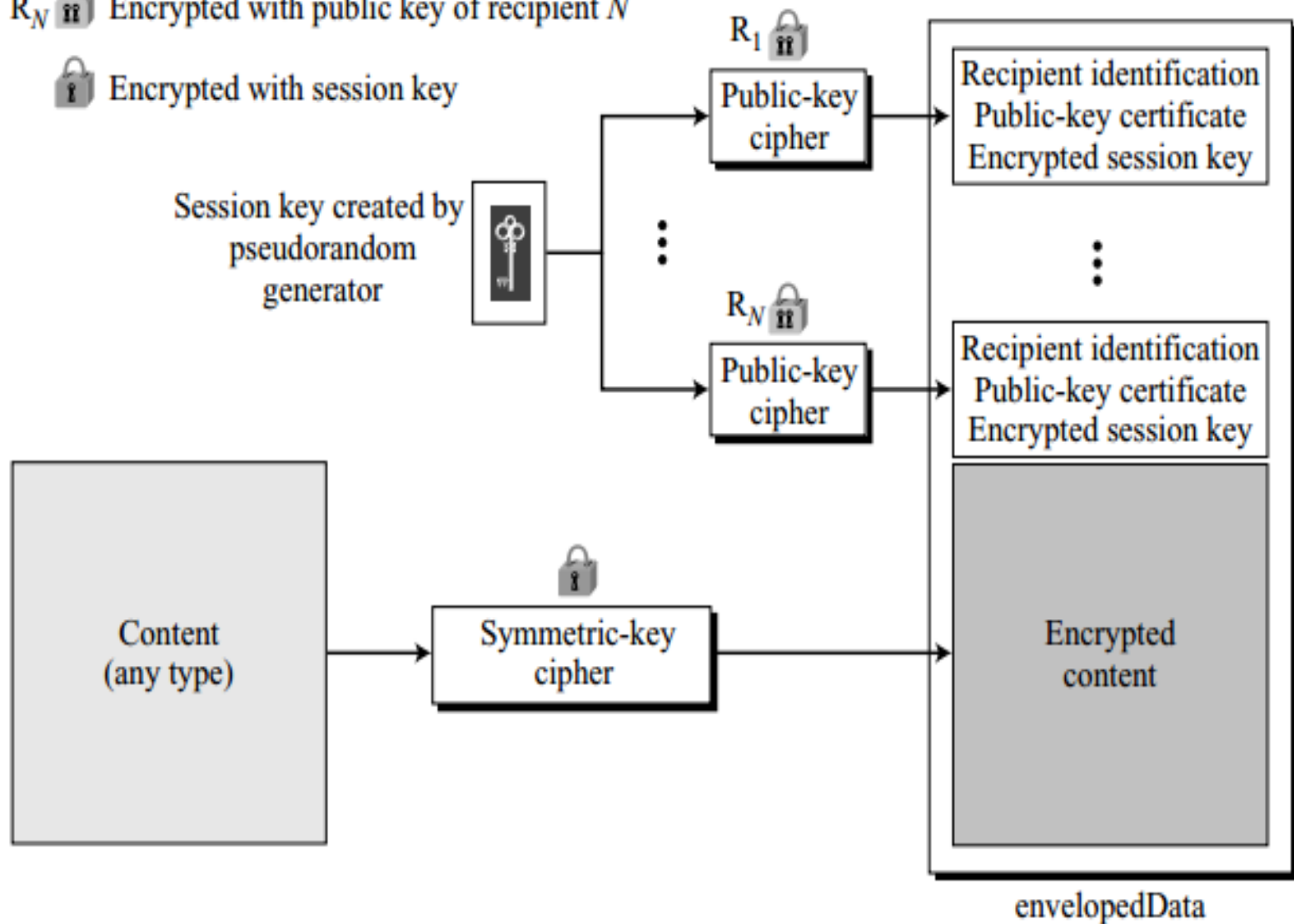
- This type is used to provide privacy for the message.
- It contains any type and zero or more encrypted keys and certificates.
- The encoded result is an object called **enveloped Data**.

R_1  Encrypted with public key of recipient 1

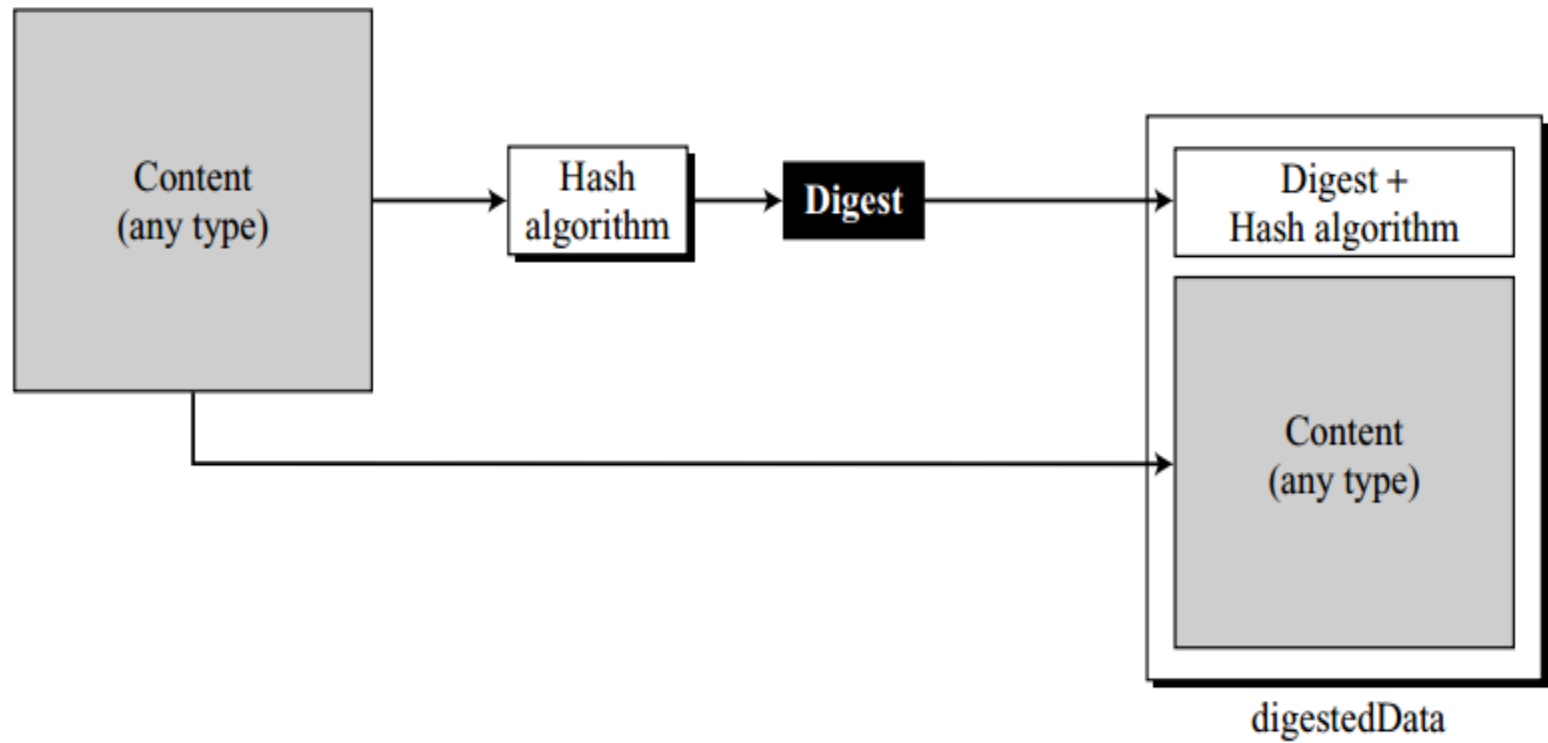
R_N  Encrypted with public key of recipient N

 Encrypted with session key

Session key created by
pseudorandom
generator



- Digested-Data Content Type
 - This type is used to provide integrity for the message.
 - The result is normally used as the content for the enveloped-data content type.
 - The encoded result is an object called **digested Data**.

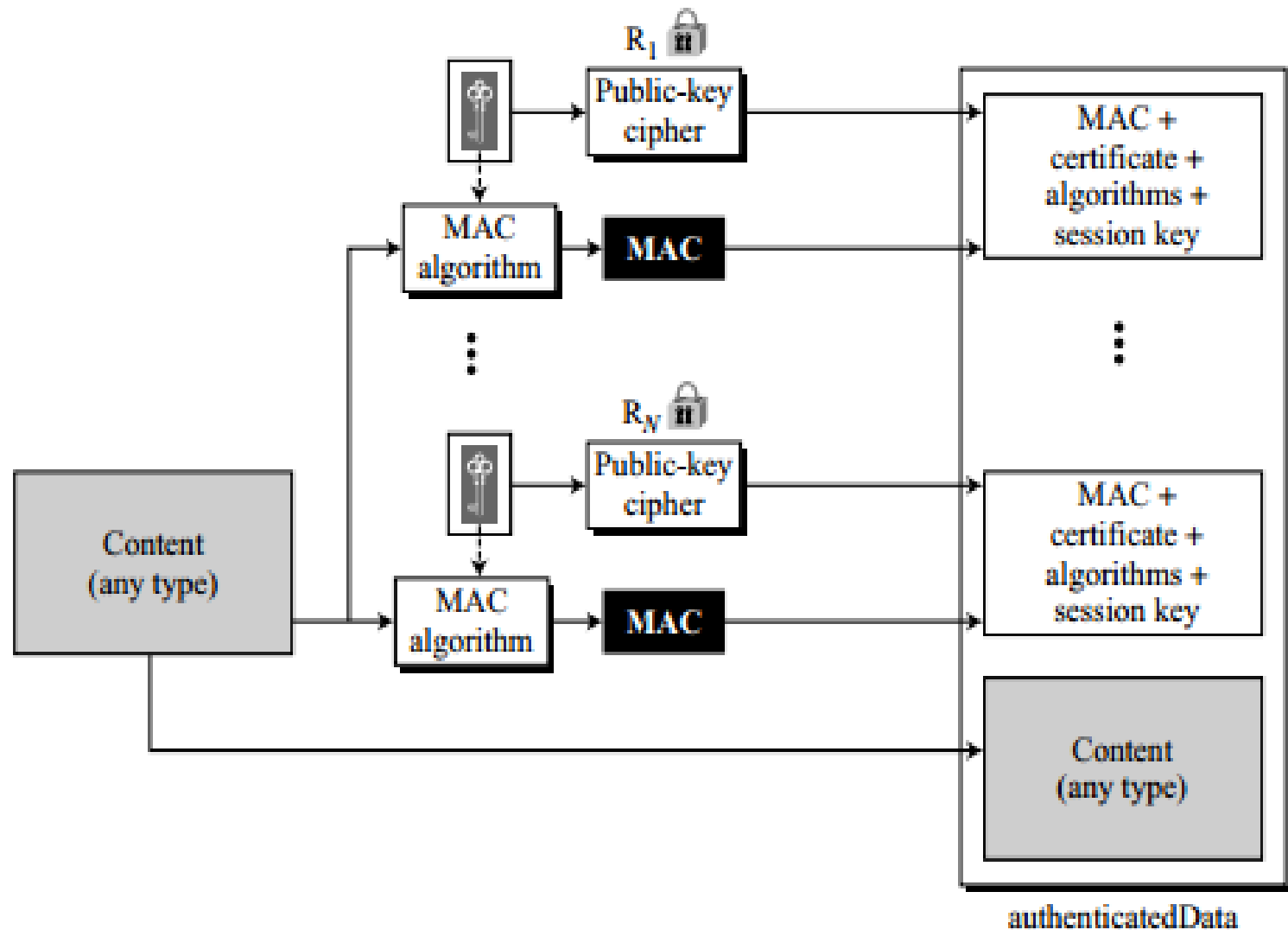


- Encrypted-Data Content Type
 - This type is used to create an encrypted version of any content type.
 - It has no recipient
 - It can be used to store the encrypted data instead of transmitting it.

- **Authenticated-Data Content Type**
 - This type is used to provide authentication of the data.
 - Using a pseudorandom generator, a MAC key is generated for each recipient.
 - The MAC key is encrypted with the public key of the recipient.
 - A MAC is created for the content.
 - The content, MAC, algorithms, and other informations are collected together to form the authenticatedData object.

R_1  Encrypted with public key of recipient 1

R_N  Encrypted with public key of recipient N



Cryptographic algorithm for S/MIME

<i>Algorithm</i>	<i>Sender must support</i>	<i>Receiver must support</i>	<i>Sender should support</i>	<i>Receiver should support</i>
Content-encryption algorithm	Triple DES	Triple DES		1. AES 2. RC2/40
Session-key encryption algorithm	RSA	RSA	Diffie-Hellman	Diffie-Hellman
Hash algorithm	SHA-1	SHA-1		MD5
Digest-encryption algorithm	DSS	DSS	RSA	RSA
Message-authentication algorithm		HMAC with SHA-1		

Key Management in S/MIME

- The key management in S/MIME is a combination of key management used by X.509 and PGP.
- S/MIME uses public-key certificates signed by the certificate authorities defined by X.509.
- User is responsible to maintain the web of trust to verify signatures as defined by PGP.